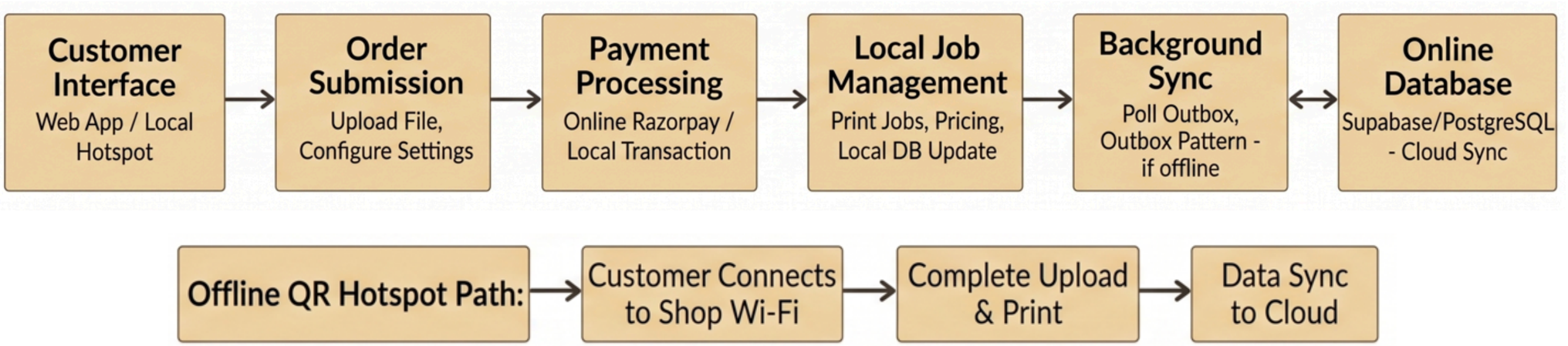
OFFSHARE

BY
Suyog Jadhav 124B1B122
Pratik Jadhav 124B1B126

PROJECT BACKGROUND: WHAT IS OFFSHARE?

- Offshare is a full-stack print shop management system that connects online ordering with offline printing.
- Upload files & customize print settings (color, copies, size, duplex)
- Secure payments via Razorpay
- Dual-mode: Cloud + Offline QR hotspot support
- Outbox Pattern ensures reliable offline-to-cloud sync
- Robust backend with 3NF database (17 tables)
- Key Importance: Ensures safe transactions, prevents data loss, and handles concurrent print jobs reliably.



ACID PROPERTIES IN OFFSHARE

Atomicity — All or Nothing (Outbox Pattern):

- When an offline job is created, the system must atomically write both the job record AND an outbox sync entry.
- If Step 2 (outbox insert) fails → automatic ROLLBACK cancels Step 1 (job insert). A job is NEVER half-created.
- On crash recovery, PostgreSQL's WAL (Write-Ahead Log) detects uncommitted transactions and auto-rolls them back.

Consistency — Integrity Constraints:

- CHECK (cost >= 0) — cost can never be negative.
- CHECK (priority BETWEEN 1 AND 10) — invalid priorities rejected.
- CHECK (status IN ('queued','printing','done','cancelled','failed')) — only valid statuses allowed.
- FOREIGN KEY (shop_id) ON DELETE RESTRICT — a shop cannot be deleted if print jobs reference it.

Isolation

- Uses SELECT ... FOR UPDATE to acquire exclusive row-level locks, preventing the Lost Update from S2.
- T2 must wait until T1 commits before it can update the same printer queue row.

Durability — Write-Ahead Log (WAL):

- When Razorpay confirms payment via webhook, PostgreSQL writes changes to the WAL on disk before applying to table pages.
- Even if server crashes 1ms after COMMIT, the WAL replays the committed transaction on reboot.
- No committed financial transaction is ever lost.

TRANSACTION CONCEPT & STATES

- A transaction is a logical unit of work that must execute completely or not at all (atomicity).
- In Offshare , a transaction might: read the printer queue → insert a new job → update payment record — all as one unit.

State	Description	Offshare Example
Active	Transaction is executing	Reading printer queue, calculating cost
Partially Committed	Final operation done, awaiting commit	Job inserted, waiting for payment confirmation
Committed	Changes permanently saved	Payment confirmed, job status set to 'queued'
Failed	Cannot proceed normally	Network failure during Razorpay callback
Aborted	Rolled back, DB restored	Job cancelled — all inserts undone via ROLLBACK
Terminated	Transaction ended	Job lifecycle complete or cancelled

TRANSACTION SCHEDULES & CONCURRENCY PROBLEM

- Scenario: Two transactions run concurrently on the printer queue (Q) for Printer P1.
  - T1 (Online Job Submission): Calculates cost, creates job record, increments queue position.
  - T2 (Offline Sync Worker): Pushes a locally submitted job to cloud, updates the same printer queue.
- Schedule S1 — Serial (Safe):
  - Execution: R1(Q) → W1(Q) → R2(Q) → W2(Q)
  - T1 reads Q=5, writes Q=6. T2 then reads Q=6, writes Q=7.
  - Result: Both jobs correctly queued. Final Q = 7. Trivially Serializable.
- Schedule S2 — Concurrent (Problematic):
  - Execution: R1(Q) → R2(Q) → W1(Q) → W2(Q)
  - Both T1 and T2 read Q=5. Both write Q=6. T1's update is overwritten (lost).
  - Result: Only one increment recorded instead of two. One print job disappears from the queue
  - This is the classic Lost Update Anomaly.

CONFLICT & VIEW SERIALIZABILITY ANALYSIS

Schedule S:

Step	T1	T2
1	R(payments)	
2		R(payments)
3	W(payments)	
4		R(print_jobs)
5	W(print_jobs)	

T1: Customer pays → updates payments table, then updates print_jobs status to paid
T2: Shopkeeper views dashboard → reads payments, then reads print_jobs

Precedence Graph: T2 → T1 (no cycle) → Conflict Serializable
T2 → T1 (Shopkeeper reads before Customer updates)

CONFLICT & VIEW SERIALIZABILITY ANALYSIS

Step	T1 (Customer uploads)	T2 (Sync worker)	T3 (Outbox logger)
1	read(file_status)		
2		read(file_status)	
3	write(file → uploaded)		
4		read(outbox_entry)	
5			read(file_status)
6		write(file → synced)	
7			write(outbox → logged)
8			write(file → archived)

Condition 1 — Initial reads: T1 is the first to read file_status (Step 1). T2 is the first to read outbox_entry (Step 4). In the serial schedule T1→T2→T3, this holds true — same transactions read the initial values first.

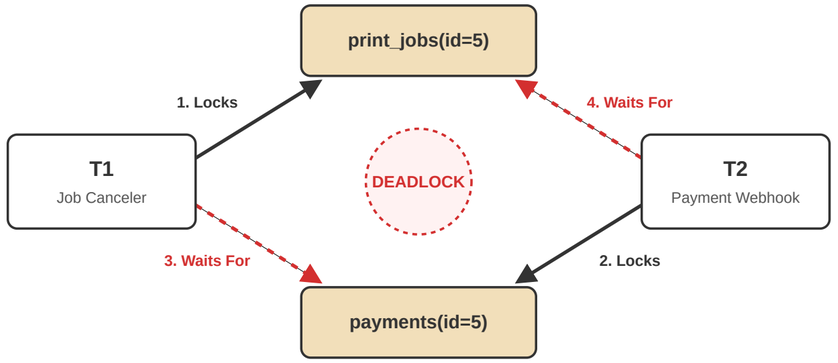
Condition 2 — Read-from relationships:
T2 reads file_status at Step 2 — this was last written by T1 at Step 3... actually T2 reads before T1 writes, so T2 reads the initial value (same as serial T1→T2→T3 where T2 runs before T3 reads T2's write). T3 reads file_status at Step 5 after T2 wrote it at Step 6 — so T3 reads from T2. This matches T1→T2→T3 serial order.

Condition 3 — Final writes: T3 performs the last write on file (Step 8) and the last write on outbox (Step 7). In serial T1→T2→T3, T3 runs last so T3 makes the final writes — this matches.
Conclusion: All 3 conditions match the serial schedule T1→T2→T3 → View Serializable.

DEADLOCK: SCENARIO & RESOLUTION

The Problem :

Step	T1 (Customer pays)	T2 (Shopkeeper updates)
1	lock(`payments`) — success	
2		lock(`print_jobs`) — success
3	lock(`print_jobs`) — **WAIT**	
4		lock(`payments`) — **WAIT**
—	stuck forever	stuck forever



The Solutions:

Prevention — Lock Ordering Rule:
Offshare enforces a global rule: always lock print_jobs BEFORE payments in every transaction.
When all transactions acquire locks in the same order, circular waits become impossible.

Detection Fallback:
PostgreSQL monitors the wait-for graph and automatically aborts the youngest transaction (victim selection) when a cycle is detected.

Timeout Strategy:
lock_timeout = '5s' is set so transactions automatically abort if they wait too long for a lock, preventing indefinite blocking.

Why NoSQL?

1. DESIGN PHILOSOPHY

DESIGNED FOR:

High Scalability
(Scale-out architecture)

Flexible Schemas
(No predefined structures)

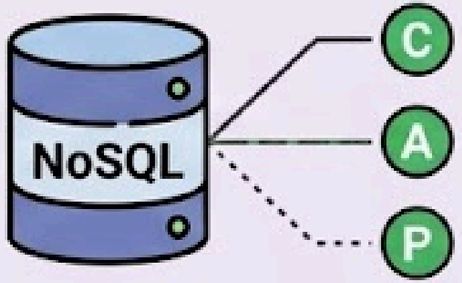
Distributed Deployment
(Multiple geographic nodes)

2. CAP TRADE-OFF

SACRIFICES:

Some ACID guarantees

To achieve Availability and Partition Tolerance



3. KEY ADVANTAGES

ADVANTAGES:

High Availability
(Uptime-focused)

Horizontal Scalability
(Add new nodes easily)

Geographic Distribution
(Deploy close to users)

Fault Tolerance
(Resilience to node failure)

4. CHALLENGES & DISADVANTAGES

DISADVANTAGES:

Consistency Challenges
(Eventual consensus)

Complex Querying
(Requires specific syntax)

Eventual Consistency
(Data syncs over time)

THANK YOU

CONFLICT & VIEW SERIALIZABILITY ANALYSIS

- Conflict Serializability of S2:Two operations conflict if they access the same data item and at least one is a write.
- Conflict pairs identified:
  - R1(Q)–W2(Q): YES → T1→T2 dependency
  - R2(Q)–W1(Q): YES → T2→T1 dependency
  - W1(Q)–W2(Q): YES → T1→T2 dependency
- Precedence Graph: T1→T2 AND T2→T1 → Cycle detected!
- Result: NOT Conflict Serializable (cycle in precedence graph proves the lost update is unsafe).
- View Serializability of S2:Checked against serial T1→T2: T2 reads initial Q in S2, but in T1→T2 serial order it must read T1's written value. Mismatch.
- Checked against serial T2→T1: T2 does final write in S2, but in T2→T1 serial, T1 does final write. Mismatch.
- Result: NOT View Serializable — S2 is not equivalent to any serial schedule. Both proofs confirm the schedule is unsafe.

Conflict Serializability — Full Explanation

What it is

A schedule is conflict serializable if it produces the same result as some serial schedule, by swapping only non-conflicting operations. Two operations conflict if they access the same data item and at least one is a write.

PrintEasy Example

The situation: Two transactions run at the same time on the printer queue value Q (queue position for Printer P1, initially Q = 5).

T1 = Online Job Submission → reads Q, then writes Q (increments queue position)
T2 = Offline Sync Worker → reads Q, then writes Q (also increments queue position)

Schedule S2 (the problematic concurrent schedule):

R1(Q) → R2(Q) → W1(Q) → W2(Q)

Step 1 — Identify conflict pairs:

PairConflict?WhyR1(Q) — R2(Q) NoBoth are reads. Read-Read never conflicts.R1(Q) — W2(Q) YesT1 reads, T2 writes same item Q → Edge: T1 → T2R2(Q) — W1(Q) YesT2 reads, T1 writes same item Q → Edge: T2 → T1W1(Q) — W2(Q) YesBoth write same item Q → Edge: T1 → T2

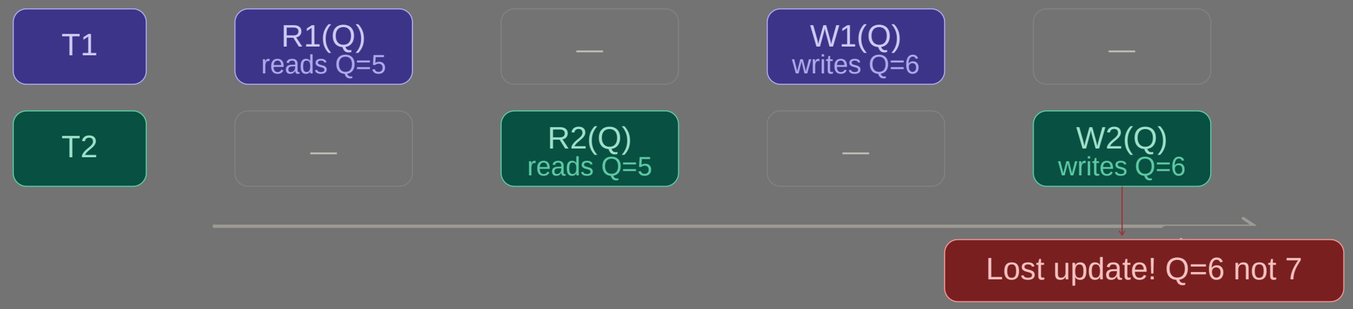
Step 2 — Draw the Precedence Graph:

T1 → T2 (from R1–W2, W1–W2) AND T2 → T1 (from R2–W1)
This creates a cycle: T1 → T2 → T1

Step 3 — Conclusion:

A schedule is conflict serializable only if the precedence graph has no cycle. Since we found a cycle, S2 is NOT conflict serializable. This formally proves that the lost update (both reading Q=5, both writing Q=6, one job lost) cannot be safely permitted.

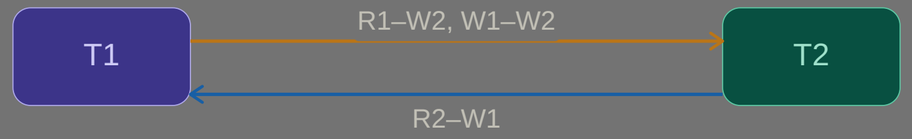
Schedule S2 — concurrent execution



Conflict pair analysis

Operation pair	Conflict?	Why	Edge
R1(Q) — R2(Q)	No	Read-Read, never conflicts	—
R1(Q) — W2(Q)	YES	T1 reads, T2 writes Q	T1 → T2
R2(Q) — W1(Q)	YES	T2 reads, T1 writes Q	T2 → T1
W1(Q) — W2(Q)	YES	Write-Write conflict on Q	T1 → T2

Precedence graph — cycle = not conflict serializable



Cycle found → NOT conflict serializable

Schedule S — three concurrent operations

	T1 — Upload	T2 — Sync worker	T3 — Outbox logger
1	read(file_status)		
2		read(file_status)	
3	write(file → uploaded)		
4		read(outbox_entry)	
5			read(file_status)
6		write(file → synced)	
7			write(outbox → logged)
8			write(file → archived)

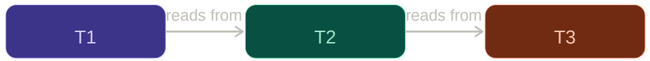
View equivalence — 3 conditions

1. Initial reads
T1 first reads file_status
T2 first reads outbox_entry

2. Read-from
T2 reads file written by T1
T3 reads file written by T2

3. Final writes
T3 final write on file
T3 final write on outbox

Deriving the equivalent serial order



All 3 conditions satisfied → View Serializable
Equivalent serial schedule: T1 → T2 → T3

Schedule S — interleaved operations

	T1 — Customer payment	T2 — Shopkeeper update
1	read(job_status)	
2		read(job_status)
3	write(payment → success)	
4		write(job_status → done)
5	read(job_status)	
6		write(payment_status)

Conflict pairs identified

Conflict 1 (W-R)
T1 write(payment) → T2 read(payment_status)

Conflict 2 (W-W)
T1 write(status) → T2 write(job_status)

Precedence graph

No cycle in graph → Conflict Serializable
Equivalent serial schedule: T1 → T2